P1214R0
Pointer to Member Functions and Member Objects are just Callables!

Published Proposal, 2018-10-06

Author:
JeanHeyd Meneide

Audience:
EWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Current Source:
github.com/ThePhD/future_cxx/blob/master/papers/source/d1214.bs

Current:
https://rawgit.com/ThePhD/future_cxx/master/papers/d1214.html

Reply To:
JeanHeyd Meneide, @thephantomderp

Abstract

This paper proposes making pointer to member objects and pointer to member functions callable.

Table of Contents

§ 1. Revision History

§ 1.1. Revision 0 - November 21st, 2014

Initial release.

§ 2. Motivation

One frequent question from beginner and intermediate users of C++ is what `&my_class::some_member_function` yields and what is the exact syntax required to use it. It's even more obscure sibling, `&my_class::some_member_variable` also garners quite a few questions of Stack Overflow and similar. The syntax is often referred to as ugly, weird, and unintuitive to both beginners and experts alike: tutorials are written just to teach what is a basic language construct, and there is special teaching that needs to be poured out into what exactly are both pointer to member functions and that they are "not regular pointers". This is even covered in Standard C++'s FAQ.

The problem grows greater in generic code, where both library developers and standard library vendors often take "callables". Sometimes it works, in the case of `std::thread`. Other times, it does not, as is the case with nearly every single standard library algorithm:

```
struct my_object {
        int value = 0;

        bool is_zero() const {
                return value == 0;
        }

        // ...
};

// ...

std::vector<my_object*> objects;
// uhhh....?
auto it = std::find_if(objects.begin(), objects.end(), &my_object::is_zero)
```

Individuals want to pass pointer to member functions and pointers to member objects, requiring quite a bit of boilerplate with either a lambda, `std::function`, `std::mem_fn`, `std::bind` or similar. This gets progressively worse because it is impossible to reference a member variable without the lambda approach or other boilerplate.

`std::mem_fn`, `std::bind`, and similar also create a severe problem with how errors are reported. One cannot just point to the top-level call to the algorithm for the error: they are bubbled up inside a deep template stack trace from the heart of `std::mem_fn` or `std::bind` or similar. Even lambdas will produce unwieldy errors, albeit compilers are better at pointing inside the lambda if the error is there rather than in the heart of `_STD _Invoke` or some other grotesque amalgamation of template spew:

> ... it would point directly to the failing call instead of failing inside the code of a mem_fn equivalent, which would be really convenient. It's worth noting that I had to fix functions several times in my library to add the missing call to as_function, which would have never been a problem with an intuitive call syntax. -- Morwenn, author of [cpp-sort]

With the introduction of `std::invoke` matching the `INVOKE` concept in the C++ standard, C++ shifted the burden of implementing callable wrappers from users to the standard library. And while this has saved many developers from writing the code themselves in a post C++17 world, there is a more

sinister underlying problem concerning frequent use of `std::invoke` by library and application developers.

While we have been saved from having 31 separate implementations of `call_wrapper` and `call_detail` and `invoke_detail`, we are still suffering from a lower-level problem that a library-based solution cannot provide to us. The amount of generated cruft that infests object files and final debug executables is extraordinarily high, and for those individuals who do not have C++17 this pain is all the more apparent when they read through their executables. Function template instantiations that perfectly forward arguments and take the functions as part of those arguments ensure that there is almost no convergence amongst the tens of thousands and hundreds of thousands of function template instantiations the compiler must generate code for, leaving each one a special template snowflake that contributes to an avalanche of additional generated compiler information. Somewhat sadly, all of this work is then ultimately discarded as unimportant in any non-debug build. The actual compile-time cost that comes from having to SFINAE or struct-specialize on this behavior is non-negligible and we are often forcing heavy-handed template constructs to do something the compiler can implement as a simple transformation.

Therefore, we propose to obviate `std::invoke` and it's pre-C++17 friends entirely by solving the very root of the problem: make the 2 oddball syntaxes for Pointer to Member Functions and Pointer to Member Objects easily callable functions with consistent syntax. This will save us both compiler resources and developer build time, and make the lives of our standard library maintainers and compiler vendors who need to make things like `std::invoke_result` and `std::invoke` work.

§ 3. Design

This design pulls from two previous papers. Peter Dimov's September 2004 N1695 - A Proposal to make Pointers to Members Callable and Barry Revzin's June 2017 p0312 - Making Pointers to Members Callable. It makes both pointer to member functions and pointer to member objects callable.

The goal of the design for this proposal is to make the simple and non-unique syntax have intrinsic and worthwhile value for both the novice and the expert programmer.

§ 3.1. But what about UFCS-

No. This is not Universal Functional Call Syntax (UFCS), has nothing to do with UFCS, and never will be UFCS. There is no correlation between fully-typed objects that are of type Pointer to Member Object or Pointer to Member Function having a specific invocation syntax and the leviathan that is

Name Lookup, the resulting Argument-Dependent Lookup (ADL) and all it implies. Pointer to member objects and pointer to member functions are fully resolved entities that do not deal with overload resolution or name lookup.

This proposal and other UFCS proposal are not in any way correlated. If anyone claims as such or attempts to use this paper as support for UFCS should be promptly directed to this section of the paper.

§ 3.2. Callable Members

In line with the goals of this proposal, we propose a very simple syntax that will allow generic code to treat all classes of "callables" -- as conceived by the `INVOKE` concept and `std::invoke` -- with a similar syntax. Particularly, the function call syntax becomes as follows:

- Function pointers, objects with `operator()`:
  `f_ptr(arg1, arg2, ..., argN);` (no change)
  `f_obj(arg1, arg2, ..., argN);` (no change)
- Pointer to member function `pmf`:
  `(obj.*pmf)(arg1, arg2, ..., argN);` can be written as `pmf(obj, arg1, arg2, ..., argN);`
- Pointer to member object `pmo`:
  `auto value = (obj.*pmo);` can be written as `auto value = pmo(obj);`
  `(obj.*pmo) = new_value;` can be written as `pmo(obj, new_value);`

N.B.: this syntax treats the arguments passed to pointer to members like any other argument, meaning that types which have conversions to `T` trigger naturally without the special exceptions or wording that is required by `INVOKE`.

§ 3.3. References, Pointers, and std::reference_wrapper, oh my!

The additional question is whether or not pointer to member functions and pointer to member objects should work with `obj` that is either a cv-qualified `T*` or a cv-qualified `T&`. For parity with `INVOKE` and `std::invoke`, and parity with `std::thread`/`std::function`/`std::bind` and other places that have `INVOKE` in the standard, it is useful to simply have the compiler automatically rewrite the syntax for pointer to member function and pointer to member functions to use the arrow `->` when `obj` is a pointer. The runtime errors if a user is dealing with an invalid pointer will be the same no matter what, and there seems little benefit here to force the user to specify.

For `std::reference_wrapper`, the implementation when calling this function would first let the natural implicit conversion from `std::reference_wrapper<T>` to `T&` happen, before the pointer to member function or pointer to member object is called. There need be no special clauses to allow this to happen, unlike for `INVOKE` and `std::invoke`.

§ 3.4. Improvement of usage with the Standard Library

The code presented in §2 Motivation now compiles without problem:

```
std::vector<my_object*> objects;
// yay!
auto it = std::find_if(objects.begin(), objects.end(), &my_object::is_zero)
```

This is an immense boon to both clarity and usability. People who perform this operation expecting a "callable" to work in this scenario have it simply work, and age-old search engine entries for "how to use member function with C++ algorithm" will dramatically decrease as code that people expect to work and have no other valid interpretation actually does exactly that!

We think lambdas have been an immense boon, but do not cover the terse and simple situations. Lambdas are great general tools to solve this problem, but feel that this paper occupies an important use case in the use of C++. Most intermediate developers who grasp the Standard Library have tried to write the code seen above with one algorithm or another: rewarding programmer intuition is a powerful way to reinforce confidence in both the language and their own skill. C++ is in a unique position in that there are many places where the language has room to reward programmer intuition: this is one such place.

§ 4. Impact

This proposal is an extension to the Core language. Its potential impacts are as follows.

§ 4.1. On the Standard

If an implementation of `std::invoke` used non-evaluated context SFINAE, that code may become ill-formed due to ambiguous overloads. The fix would be to delete the offending overloads for when this paper is adopted, and to date we do not know of any library vendors that have objection to making this fix. It is also easy to do because Feature Test Macros are part of the C++ standard now.

There are no backwards compatibility compromises or breaks for this feature in the standard. The usage of parenthesis and function call syntax in this case do not conflict or produce grammar or parsing ambiguities or errors.

§ 4.2. On User Code

The proposal enables syntax that was not previously enabled. This may only break code which relied on the fact that certain non-evaluated context SFINAE's that implemented their own version of `INVOKE`/`std::invoke`. If a set of overloads used said non-evaluated context-style SFINAE (e.g., `decltype()` SFINAE) and that set of overloads relied exactly on the differences between pointer to member object syntax versus pointer to member function syntax versus regular function call syntax, those set of overloads might be made ambiguous.

The good news is that the fix is easy: delete the offending overloads since only 1 is valid. In many cases, deleting the function altogether would be a plausible fix and just invoking the callable directly!

There are also no known cases where the intent of the code changes or the behavior changes into a form that will silently break code, and all breakages due to the above are loud and easily-fixable compile-time errors.

§ 5. Proposed Wording and Feature Test Macros

The following wording is relative to [n4762].

§ 5.1. Proposed feature Test Macro

The recommended feature test macro is `__cpp_invokable_members`.

§ 5.2. Intent

The intent of this wording is to add `pointer_to_member_function(obj, arg1, arg2, ..., argN)`, `auto value = pointer_to_member_object(obj)`, and `pointer_to_member_object(obj, arg1)` as valid expressions. The last expression would be ill-formed if the `pointer_to_member_object` itself refers to a `const` member or the object itself is `const`. Similarly, neither `pointer_to_member_function` or `pointer_to_member_object`

would work with an `obj` that does not match the required cv- and reference-qualifiers on that function (this is not different from how it works currently, but the wording must match).

§ 5.3. Proposed Wording

Modify §7.6.1.2/1 [**expr.call**]/1 to read as follows:

> [1] A function call is a postfix expression followed by parentheses containing a possibly empty, comma-separated list of initializer-clauses which constitute the arguments to the function. The postfix expression shall have ~~function type or function pointer type~~ function type, function pointer type, or pointer-to-member type . For a call to a non-member function or to a static member function, the postfix expression shall be either an lvalue that refers to a function (in which case the function-to-pointer standard conversion (7.3.3) is suppressed on the postfix expression), or it shall have function pointer type.

Add one clause after §7.6.1.2/2 [**expr.call**]/2:

> [3] For a call of the form `pm(a1, ..., aN)`, where `pm` is of type "pointer to member of `T`".
>
> — if `pm` is a pointer to member function taking `M` arguments, then `N` shall be `1+M`. The result of the expression shall be equivalent to calling a pointer to member function (7.6.4) with the syntax `(a1.*pm)(a2, ..., aN)` if `a1` is a possibly cv-qualified class type of which `T` is a base class. Otherwise, the behavior of the expression shall be equivalent to `(a1->*pm)(a2, ..., aN)`.
>
> — if `pm` is a pointer to data member, then either
>
> — `N` shall be `1`. The behavior of the function call shall be as-if invoking a pointer to member data (**7.6.4**) with `a1.*pm` if `a1` is a possibly cv-qualified class type of which `T` is a base class or is convertible to such a class type, `a1->*pm` otherwise. Or,
>
> — `N` shall be `2`. The behavior of the function call shall be as-if assigning the second argument `a2` to the result of a pointer to member data (**7.6.4**) with `(a1.*pm) = a2` if `a1` is a possibly cv-qualified class type of which `T` is a base class or is convertible to such a class type, `(a1->*pm) = a2` otherwise.

Append to §14.8.1 Predefined macro names [**cpp.predefined**]'s **Table 16** with one additional entry:

Macro name	Value
__cpp_invokable_members	201811L

6. Acknowledgements

Thank you to Jason Turner for showing me some of the internals of ChaiScript a long time ago and bringing this common problem to light. Thank you to Stephan T. Lavavej for talking about this during one of his talks. Thank you to Barry Revzin and Peter Dimov for their previous work and scholarship on this matter.

References

Informative References

[CPP-SORT]
Morwenn. cpp-sort. October 7th, 2018. URL: https://github.com/Morwenn/cpp-sort

[N1695]
Peter Dimov. A Proposal to Make Pointers to Members Callable. 9 September 2004. URL: https://wg21.link/n1695

[N4762]
ISO/IEC JTC1/SC22/WG21 - The C++ Standards Committee; Richard Smith. N4762 - Working Draft, Standard for Programming Language C++. May 7th, 2018. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/n4762.pdf

[P0312]
Barry Revzin. Making Pointers to Members Callable. October 12th, 2018. URL: https://wg21.link/p0312

[pmf-CG]
CodeGuru. C++ Tutorial: Pointer to Member Function. June 30th, 2010. URL: https://www.codeguru.com/cpp/cpp/article.php/c17401/C-Tutorial-PointertoMember-Function.htm

[PMF-ISO]
C++ Standard Foundation. Pointers to Member Functions. 2018. URL: https://isocpp.org/wiki/faq/pointers-to-members

[pmf-SO]
Johannes Schaub - litb. Function pointer to member function. March 8th, 2010. URL: https://stackoverflow.com/a/2402607

[pmf-SO-algo]

Piotr Skotnicki. Passing C++ Member Function Pointer to STL Algorithm. May 20th, 2015. URL: https://stackoverflow.com/a/30355058